AS-BUILT PROJECT REPORT

# PosterScore AI

How the application works, how it was built, how the analysis logic is separated, and how the product evolved from the first UI prototype to the current evidence-led MVP.

| UPLOAD | MEASURE | ANALYZE | IMPROVE |
|---|---|---|---|

**PosterScore AI is a full-stack poster review tool. The browser handles the experience, the server validates and measures the image, Gemini provides visual interpretation, and deterministic application code owns the final score, rule caps, reliability checks, and user-safe errors.**

## Current local setup

```
Frontend: http://localhost:5173/
Backend API: http://localhost:8787
Combined command: npm run dev
UI-only command: npm run dev:ui
```

Report date: 30 October 2026. This document describes the source currently in the project folder and does not include or reproduce any API key.

# 1. Product purpose and user workflow

The product is designed for a user who has a poster draft and wants a direct creative-direction review. The app is not just a chatbot that returns compliments. It first checks whether the upload is actually a poster, gathers objective image facts, asks Gemini for visual interpretation, validates the result, calculates a score in code, and turns the findings into a prioritized improvement plan.

## The visible workflow

| Stage | What the user sees | What the application does |
|---|---|---|
| 1. Upload | Drag and drop or choose a JPG, PNG, or WEBP poster. Add platform, goal, audience, or brand context. | Checks file type and size in the browser, creates a preview, and keeps the context ready for the API. |
| 2. Diagnose | A staged loading screen shows the poster preview, scan animation, and named review stages. | The server decodes, measures, preprocesses, classifies, runs rules, calls Gemini, and validates the response. |
| 3. Improve | The report shows a score, category bars, evidence checks, strengths, and prioritized fixes. | The server computes the final score and the UI lets the user filter fixes, inspect markers, review history, and print. |

## Non-poster protection

If Gemini classifies the image as not a poster, the app does not show misleading category scores. It renders a dedicated explanation, the uploaded image, and an Analyze another image action. This directly addresses the requirement that the system distinguish a poster from a random image before judging design quality.

## The reliability boundary

**The report explicitly says that structural feedback on legibility, hierarchy, spacing, and clarity is more dependable than subjective taste about style or concept. Style and concept comments are suggestions, not objective facts; calibration data should be used to measure where the AI is reliable.**

# 2. Technology stack and architecture

The implementation uses a small full-stack architecture with a clear division of responsibility. React and Vite own the browser experience. Express owns the API boundary. Sharp owns safe image handling. Gemini owns visual interpretation. Zod, the rule registry, and the scoring functions keep the model output bounded and explainable.

| Layer | Technology | Role in this app |
|---|---|---|
| Browser UI | React 19 + TypeScript | Component state, upload flow, loading stages, result report, filters, version history, and print action. |
| Frontend tooling | Vite 6 | Development server on port 5173, API proxy to port 8787, and production bundling. |
| API server | Node.js + Express 5 | Multipart upload route, CORS, rate limiting, request IDs, health endpoint, and safe error responses. |
| Upload handling | Multer memory storage | Receives the image without persisting it to disk and enforces the request file limit. |
| Image processing | Sharp | Decodes the file, checks dimensions, rotates by EXIF, measures image facts, resizes, flattens, and creates a JPEG for Gemini. |
| AI provider | Google Gemini via @google/genai | Classifies poster versus non-poster and produces visual evidence, category checks, critique, and recommendations. |
| Validation | Zod | Defines the allowed analysis shape and rejects malformed or incomplete model output. |
| Testing | Vitest | Covers scoring, rules, validation, provider reliability, upload safety, correction loops, and API response guarding. |

## High-level architecture

**React + Vite browser -> multipart image and JSON context -> Express API -> Multer and Sharp -> measurements and rules -> Gemini provider -> Zod schema and quality-control loop -> deterministic score -> JSON report -> React results UI.**

The Gemini API key is server-only. It is read from GEMINI_API_KEY in the server environment and is never placed in a VITE_ variable or sent to the browser.

# 3. Source structure and responsibility

The source is organized so that the UI does not have to know how the provider works and the provider does not decide the final score. The following map describes the important files in the current implementation.

| File | Responsibility |
|---|---|
| src/App.tsx | Main React state machine: home, loading, and results views; upload handlers; context fields; score report; rule checks; critique; history; print action. |
| src/analysisApi.ts | Frontend API boundary. Sends multipart image and context, accepts only a validated response shape, and converts server failures into user-facing request errors. |
| src/types.ts | Shared browser types for analysis, categories, measurements, improvements, element maps, critique, and rule results. |
| src/styles.css | Product visual system, responsive grids, upload card, loading screen, report sections, print rules, and mobile refinements. |
| server/app.ts | Express app, CORS, request IDs, upload route, health route, rate limiter, demo branch, analysis orchestration, and provider error mapping. |
| server/validation.ts | Allowed formats, 10 MB limit, maximum dimensions and pixels, Sharp decode checks, preprocessing, and deterministic image measurements. |
| server/vision.ts | Gemini request construction, system and user prompts, structured response handling, retry options, and provider invocation. |
| server/provider.ts | Provider error classification, Retry-After parsing, retryable categories, exponential backoff with jitter, and ProviderError. |
| server/rules.ts | Facts model, deterministic rule registry, rule basis labels, skip behavior, evidence, targets, and failed-rule helpers. |
| server/analysis.ts | Schema, category definitions, type-specific weights, score calculation, caps, rating labels, quality checks, correction loop, and demo analysis. |
| server/validation.ts + server/app.ts | Together they prevent unsafe files, malformed context, oversized images, and unhelpful API failures from reaching the model or user. |
| tests/*.ts and src/analysisApi.test.ts | Fixtures and tests for deterministic behavior. Current run: 27 passed and 4 skipped tests across 4 test files. |

**The file boundaries make the product easier to tune: UI changes stay in src, scoring changes stay in server/analysis.ts, measurable design policy stays in server/rules.ts, and provider changes stay in server/vision.ts and server/provider.ts.**

# 4. Upload, validation, and image facts

The upload pipeline is deliberately defensive because the AI should not be asked to interpret corrupt, unsupported, oversized, or unsafe image data.

## Validation sequence

| Check | Current logic |
| --- | --- |
| Browser acceptance | The file input accepts image/jpeg, image/png, and image/webp. The same handler supports click selection and drag-and-drop. |
| MIME and extension | The server requires an allowed MIME type and .jpg, .jpeg, .png, or .webp extension. |
| File size | The hard limit is 10 MB. Multer rejects oversized request bodies and the validation layer repeats the safety check. |
| Decode | Sharp must read width and height. Corrupt or non-image bytes receive a clear image-processing error. |
| Pixel safety | Images above 12,000 pixels on either axis or 40 million total pixels are rejected before deeper processing. |
| Preprocessing | Sharp rotates according to EXIF, fits inside 2400 by 2400 without enlargement, flattens transparency against the product background, and produces a quality-88 JPEG. |

## Measured image facts

+ Width, height, aspect ratio, and orientation: portrait, landscape, or square.
+ Dominant colors: a deterministic 96 by 96 sample grouped into coarse RGB buckets.
+ Estimated whitespace: percentage of sampled pixels that are near white.
+ Estimated image coverage: a contrast-edge estimate from horizontal and vertical luminance differences.

**These are evidence signals, not claims of exact OCR or layout detection. The product labels them as estimates and keeps unavailable measurements out of the score rather than inventing them.**

# 5. Gemini analysis and critique logic

Gemini is used for visual interpretation, not as the sole source of truth. The model receives the prepared image, user context, and a strict output contract. The server then validates the answer and can ask for a correction if the response conflicts with the contract.

## What Gemini is asked to produce

| Output | Purpose |
| --- | --- |
| isPoster and posterType | Protects the app from scoring a random image as a designed poster and selects the supported type profile. |
| posterPurpose and confidence | Explains the likely job of the poster and how confident the review is. |
| firstNotice, secondNotice, thirdNotice | The three-second test: what the eye notices first, second, and third. |
| elementMap | Stable element IDs such as headline, CTA, image, supporting copy, or brand mark for grounded feedback. |
| categories | Seven categories, each with 2 to 4 named checks, 0 to 10 scores, strengths, issues, and evidence. |
| critique | Intent, root causes, one big move, up to three top fixes, keep, optional alternative directions, and polish. |
| improvements | High, medium, or low priority fixes with problem, evidence, why it matters, recommendation, and optional marker coordinates. |

## Quality-control loop

1. The candidate is parsed with Zod. 2. The server checks poster/non-poster consistency, duplicate evidence, target IDs, specificity, generic language, critical-rule coverage, alternative-direction requirements, CTA contradictions, and score inflation. 3. If checks fail, the exact issues are sent back to Gemini for a correction. 4. The loop allows a maximum of two correction passes after the first answer. 5. If the answer still fails, the API returns a controlled failure instead of silently showing unreliable feedback.

**This is why the UI can show an actual provider or validation reason instead of always saying only 'try another image'.**

# 6. Deterministic design rules

The rules layer is the main protection against generic or inflated AI feedback. It owns measurable pass/fail evidence, target element IDs, severity, score caps, and honest skipped states.

| Rule area | Examples in the registry | Effect |
|---|---|---|
| Hierarchy | Dominant headline, focal saliency, distinct levels, headline length. | Checks whether the intended first read is visibly dominant. |
| Composition | Safe margin, balance, proximity, spacing consistency, collisions, text over subject, alignment mixing. | Flags measurable structural problems without asking Gemini to invent coordinates. |
| Typography | Maximum font families, weight limit, line height, line length. | Turns typographic rules of thumb into inspectable evidence. |
| Readability | Contrast, long all-caps, busy background, centered paragraphs. | Protects legibility and applies hard caps to serious readability failures. |
| Color | Palette restraint, accent coverage, text-color count. | Keeps color feedback tied to measurable palette behavior. |
| CTA and branding | Single primary CTA, action wording, CTA distinctness, logo size, logo clear space. | Adapts the review to poster jobs where action or brand recognition matters. |
| Clarity | Word budget, essentials, spelling, image sharpness. | Checks whether the viewer can understand and act on the poster. |

## Basis labels

+ standard: a published specification or standard, such as contrast-related guidance.
+ principle: a widely taught design principle, such as proximity or alignment.
+ heuristic: a starting threshold chosen for this product and intended to be calibrated on real samples.

**When detection confidence is below 0.6, layout rules are skipped. If a required fact such as subject, sharpness, or accent coverage is unavailable, the related rule is shown as not evaluated. Skipped rules do not count as failures or deductions.**

# 7. Scoring logic and score integrity

The application deliberately separates AI judgment from arithmetic. Gemini supplies category checks and evidence, while the server calculates category scores, applies type-aware weights, handles non-applicable branding, applies rule caps, and derives the final rating.

## Category calculation

For each category, the server averages the 0 to 10 check scores and converts the average to a 0 to 100 category score. Scores are clamped to the 0 to 100 range. The seven categories are Layout & hierarchy, Typography, Readability & contrast, Imagery / graphics, Message & CTA, Color, and Branding.

## Poster-type weighting

| Poster type | Emphasis in the current profile |
|---|---|
| Promotional Offer | Layout 20%, Message & CTA 20%, Branding 15%; other categories share the remaining weight. |
| Event | Layout 20%, Typography 15%, Readability 15%, Imagery 15%, Branding 15%. |
| Restaurant/Food | Imagery 25%, Layout 15%, Readability 15%, Branding 15%; other categories receive smaller weights. |
| Corporate | Typography 20%, Branding 20%, Readability 15%, Imagery 15%, Layout 15%. |
| Other supported types | A balanced base profile: Layout 20%, Typography 15%, Readability 15%, Imagery 15%, Message 15%, Color 10%, Branding 10%. |

## Conditional branding

When brandingApplicable is false, the branding weight becomes zero and its weight is redistributed proportionally across the remaining categories. This prevents a poster from being penalized for a dimension that is not judgeable.

## Rule caps and rating labels

**A failed rule with a score cap can lower the corresponding category even when the draft model score is higher. The final score is the weighted category result, rounded and clamped to 0 to 100. Rating labels are: Exceptional at 90+, Strong Design at 80+, Good Design at 70+, Needs Improvement at 60+, Weak Design at 40+, and Major Redesign Needed below 40.**

# 8. Frontend screens and interaction logic

The frontend is a compact state machine in src/App.tsx. It moves between home, loading, and results views and keeps the current File, preview URL, analysis, error, retry state, loading stage, context, and browser history in React state.

| UI area | Implementation details |
| --- | --- |
| Landing page | Hero headline, upload card, product promise, three-step explanation, design-system principles grid, and responsive footer. |
| Upload area | Drag events set a visual dragging state. File selection and drop both call the same file validation and preview path. The selected file can be removed before analysis. |
| Context fields | Target platform is a select with General, Instagram post, Instagram story, A3 print, WhatsApp status, and Billboard. Goal, audience, and brand information are optional. |
| Loading page | A 500 ms interval advances through eight named stages while the request runs. The scan line and progress bar provide a visible sense of progress. |
| Results page | Poster results show score ring, rating label, confidence, limitations, intended use, three-second test, measurements, rules, category evidence, critique, version history, and next-analysis CTA. |
| Non-poster result | The score UI is replaced by an explanation and a new-analysis action. This prevents false precision. |
| Improvement filters | High, medium, and low priority tabs select the visible improvement detail. Validated anchor coordinates can place numbered markers over the poster. |
| History and export | Up to five reports are stored in localStorage. Export uses window.print and CSS print rules rather than server-side PDF generation. |

## Responsive work completed

+ Desktop uses multi-column hero, analysis, report, and category layouts.

+ Tablet collapses major sections and keeps the upload card within the viewport.

+ Mobile switches to single-column sections, flexible filenames, compact category rows, full-width actions, and safer spacing.

+ The principles grid uses complete card boundaries, keeps the Color divider visible, and intentionally removes the final Branding bottom rule as requested.

# 9. API behavior, errors, and Gemini reliability

The Express API has two public routes and a controlled failure model. Errors return a stable code, safe message, retryable flag, and request ID. Server logs include request ID, provider, model, duration, and error category without logging image data or credentials.

| Endpoint | Behavior |
| --- | --- |
| GET /api/health | Returns API status, demo mode, provider name, configured model, fallback-model presence, and whether a Gemini key is configured. It never returns the key. |
| POST /api/analyze | Accepts multipart image data plus a JSON context field. Validates the file and context, measures the image, calls the provider, runs rules and quality control, then returns a final analysis and request ID. |

## Provider error categories

+ AUTH: the key is invalid or rejected.
+ BILLING or QUOTA: the provider requires permission, billing, or available quota.
+ RATE_LIMIT: the request frequency is temporarily too high.
+ UNAVAILABLE: the provider is busy or returns a transient service failure.
+ NETWORK: timeout, aborted request, or connection failure.
+ MODEL, INVALID_REQUEST, and RESPONSE_FORMAT: non-transient configuration or response problems.

## Retry behavior

**Transient rate-limit, unavailable, and network failures can be retried with bounded exponential backoff, optional Retry-After support, a maximum delay, and jitter. Permanent authentication, billing, model, and invalid-request failures are not retried endlessly. The user sees a message that points to the likely cause instead of a generic image failure.**

# 10. How the project changed from the starting point

**The folder does not contain a usable Git history for a commit-by-commit comparison, so this is a reconstructed implementation timeline from the project state and the requested product changes. It describes the meaningful product phases rather than claiming exact commit dates.**

| Phase | Starting behavior or request | Change now present |
|---|---|---|
| 1. UI foundation | Build the visual workflow before configuring an API. | Created the React/Vite landing page, upload card, drag-and-drop preview, staged loading view, report layout, mobile styling, and demo/mock path. |
| 2. Real provider migration | Use Gemini instead of the earlier provider direction. | Moved the server provider to @google/genai and GEMINI_API_KEY, added model configuration, health reporting, and server-only secret handling. |
| 3. Failure diagnosis | The UI showed 'We couldn't complete the review' without saying why. | Added provider error classification, request IDs, retry flags, clearer auth/billing/quota/rate-limit/model/network messages, and bounded retry behavior. |
| 4. Slow and stuck analysis | The loading screen could remain ambiguous while Gemini was slow or unavailable. | Added meaningful loading stages, retries for transient failures, fallback model configuration, timeout-aware errors, and validation failure reporting. |
| 5. Poster detection | The AI feedback could be wrong for a non-poster image. | Added explicit isPoster classification and a dedicated non-poster result that withholds design scoring. |
| 6. Same scores and generic feedback | Category scores could be too similar and recommendations could be vague. | Added evidence checks, minimum 2 and maximum 4 checks per category, duplicate detection, generic-advice rejection, score integrity checks, and type-aware weights. |
| 7. Rule-based grounding | Subjective model judgment needed measurable guardrails. | Added a deterministic rule registry with standard, principle, and heuristic bases, target IDs, severity, caps, evidence, and not-evaluated states. |
| 8. Art-director critique | Feedback needed to be specific and useful to a designer. | Added intent, audience, desired action, three-second test, root causes, one big move, up to three quantified fixes, keep, polish, and low-score alternative directions. |
| 9. Responsive refinement | The UI had white screens, spacing problems, and uneven section boundaries during visual testing. | Refined responsive breakpoints, safer widths, mobile stacking, file-name wrapping, report layout, and the principles card dividers, including the final Branding rule removal. |
| 10. Verification | The MVP needed repeatable confidence instead of visual inspection alone. | Added 27 passing and 4 skipped automated tests across 4 files, plus a passing TypeScript/Vite production build. |

The architectural direction stayed intentionally stable: browser UI, Express API, Sharp, Gemini, deterministic rules, validation, scoring, and a report. The changes improved reliability and specificity without adding a database, authentication, or cloud storage system.

# 11. Testing, commands, and deployment notes

The current project can be run locally in either a full-stack mode or a UI-only/demo mode. The tests use mocked providers and in-memory fixtures so the scoring and reliability logic can be verified without spending Gemini quota.

## Commands

```
npm install
npm run dev # Vite frontend plus Express backend
npm run dev:ui # Vite frontend only
npm test
npm run build
```

## Demo mode

```
VITE_USE_MOCK_ANALYSIS=true DEMO_MODE=true npm run dev
```

The browser mock keeps the loading, report, filters, markers, history, and print flow testable without an API key. The server demo branch can also return a deterministic report when DEMO_MODE=true and no Gemini key is configured.

## Current verification

| Check | Current result |
| --- | --- |
| Vitest | 4 test files passed; 27 tests passed and 4 tests skipped. The skipped cases are the optional socket-bound integration paths in the managed environment. |
| Production build | TypeScript project build and Vite production bundle completed successfully. |
| Secrets | The report and source inspection do not expose the configured Gemini key. |
| Browser behavior | The in-app browser can load the running frontend at http://localhost:5173/. |

## Production checklist

+ Use a Gemini model ID confirmed as enabled for the project.
+ Keep GEMINI_API_KEY server-side and do not commit .env.
+ Set a production CORS allowlist and HTTPS reverse proxy.
+ Replace the in-memory rate limiter with a durable shared limiter when running multiple instances.
+ Run npm test and npm run build in CI before deployment.

# 12. Current limitations and next logical improvements

The MVP is intentionally honest about what it can and cannot measure. The main opportunity is not to add more adjectives to the AI prompt; it is to add reliable detectors and calibration data so more rules can move from not evaluated to evidence-backed.

| Current limitation | Why it matters | Logical next step |
|---|---|---|
| No deterministic OCR/layout detector | Many element-level rules need text boxes, font metrics, subject regions, and saliency. | Add an OCR and layout extraction stage that produces stable element IDs and confidence values. |
| Whitespace and coverage are estimates | They are sampled image facts, not exact semantic layout measurements. | Compare estimates against labeled poster fixtures and calibrate thresholds. |
| Gemini taste is subjective | Style and concept judgments may not match an expert designer. | Collect calibration ratings from human reviewers and show reliability by criterion. |
| RTL and Arabic are not specialized | English-first CTA and spelling logic can misjudge correct non-English posters. | Add direction-aware layout checks, Arabic CTA verbs, and a language-aware spell checker. |
| No persistent account history | History is local to one browser and is not shared across devices. | Add authentication and a privacy-reviewed database only if the product requires it. |
| Browser print is the PDF export | Layout depends on the browser print dialog and local print settings. | Add a dedicated export renderer if consistent downloadable reports become a product requirement. |

## Final summary

**PosterScore AI now works as a controlled analysis pipeline rather than a single unconstrained AI call. The browser provides a clear workflow, the server protects image and provider boundaries, the rules engine supplies measurable facts, Gemini supplies interpretation, validation rejects weak answers, and deterministic code owns the final score. That separation is the central design decision that makes the MVP explainable, testable, and ready for calibration.**